

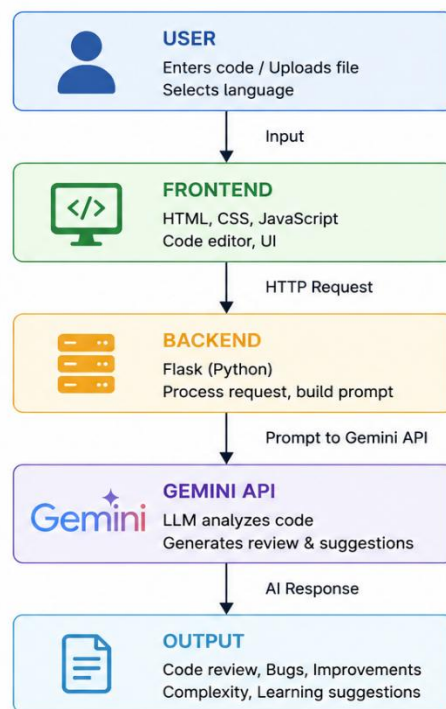
DevMentor AI

1. Overview

DevMentor AI is an AI-powered code review and learning assistant built for students and early-career developers. The goal of the system is to help users improve code quality, identify bugs, understand complexity analysis, and receive personalized learning recommendations.

Instead of manually reviewing code, users can submit source code through a web interface. The system sends the code to Google's Gemini Large Language Model (LLM), which analyzes it and generates detailed feedback. The application combines a responsive frontend, a Flask backend, and Gemini AI to create a complete AI-assisted learning workflow.

2. System Architecture



DevMentor AI follows a three-layer architecture consisting of a Frontend, Backend, and AI Processing Layer. The frontend is built using HTML, CSS, and JavaScript and provides an interactive interface where users can paste code, upload source files, select a programming language, and view analysis results. Once the user submits the code, the frontend sends the code and selected language to the backend through an HTTP POST request. The backend is developed using Python Flask and acts as the central processing layer of the system. It receives the user input, validates the request, and constructs a structured prompt for the AI model. The backend

then communicates with the Gemini API by sending the generated prompt and waits for the response. After receiving the AI-generated analysis, the backend processes the response and returns it to the frontend in JSON format.

The AI layer is powered by Google's Gemini Large Language Model. This layer performs the core intelligence of the application by analyzing the submitted source code, identifying bugs, suggesting improvements, generating corrected code, estimating time and space complexity, and providing learning recommendations. The generated feedback is then sent back through the backend to the frontend, where it is displayed to the user in a clear and organized format.

The overall workflow of the system follows a simple Input → Process → Output pattern. The user provides source code as input, the backend and Gemini API process the code, and the system outputs a detailed AI-generated review along with learning suggestions and code statistics. This architecture ensures modularity, scalability, and easy maintenance while demonstrating the practical integration of Large Language Models into software development tools.

3. Workflow (Input → Process → Output)

3.1 Input Stage

The workflow begins when the user interacts with the DevMentor AI interface. The user can either paste source code directly into the code editor or upload a source code file. After selecting the appropriate programming language, the user clicks the **Run Review** button to start the analysis process. At the same time, the frontend performs basic code statistics calculations and displays them instantly to improve the user experience.

The frontend calculates and displays:

- Number of lines
- Character count
- Selected programming language

These statistics provide users with quick insights about their code before the AI review begins.

3.2 Processing Stage

Once the review request is submitted, the frontend collects the user's code and programming language and sends the data to the Flask backend through an HTTP POST request in JSON format. The backend receives the request and dynamically constructs a structured prompt that instructs the AI model to analyze the code and generate meaningful feedback.

The generated prompt requests the AI to provide:

- Bugs Found
- Improvements
- Corrected Code

- Time Complexity
- Space Complexity
- Learning Recommendations

After the prompt is created, the backend sends it to the Gemini API. The Gemini Large Language Model performs the core analysis by understanding the source code, detecting potential bugs, evaluating code quality, estimating complexity, and generating personalized learning recommendations. Once the analysis is completed, Gemini returns a detailed response to the backend. The backend then processes the response and converts it into a structured JSON format that can be easily displayed by the frontend.

3.3 Output Stage

After the Gemini API completes the code analysis, the generated response is sent back to the Flask backend. The backend processes the response and returns it to the frontend in JSON format. The frontend then displays the results in an organized and user-friendly manner, enabling users to easily understand the feedback and improve their code.

The primary output of the system is the **AI Code Review**, which provides detailed insights into the submitted code. The review identifies syntax errors, logical mistakes, inefficient coding practices, and potential improvements. When applicable, the system also suggests a corrected version of the code and explains why the changes are necessary.

The output generated by DevMentor AI consists of the following components:

AI Review

- Identification of bugs and errors
- Code quality assessment
- Suggestions for improvement
- Corrected or optimized code snippets

Complexity Analysis

- Time Complexity estimation
- Space Complexity estimation
- Performance-related observations

Learning Recommendations

- Topics the user should study to strengthen understanding
- Best coding practices related to the submitted code
- Suggestions for improving programming skills

Code Statistics

- Total number of lines
- Character count
- Selected programming language

By presenting both technical feedback and educational guidance, the system not only helps users fix errors but also supports continuous learning and skill development. This transforms DevMentor AI from a simple code-review tool into an AI-powered learning assistant that provides actionable feedback in real time.

4. Prompt Design

Prompt design is one of the most important components of DevMentor AI because it determines the quality and structure of the responses generated by the Gemini Large Language Model. Instead of sending only the user's source code to the AI, the backend creates a carefully structured prompt that clearly specifies the type of analysis required. This approach helps ensure that the generated output is consistent, organized, and relevant to the user's needs.

When a user submits code, the Flask backend dynamically inserts the selected programming language and source code into a predefined prompt template. The prompt instructs Gemini to perform a comprehensive code review and provide feedback in a structured format.

Example Prompt:

Review the following Python code.

<USER CODE>

Return:

1. Bugs Found
2. Improvements
3. Corrected Code
4. Time Complexity
5. Space Complexity
6. Learning Recommendations

The prompt is designed to guide the AI through multiple analysis tasks simultaneously. First, it identifies syntax errors, logical mistakes, and potential bugs in the code. Next, it evaluates code quality and suggests improvements based on programming best practices. The model then generates a corrected version of the code when necessary and estimates the time and space complexity of the solution. Finally, it provides learning recommendations to help users improve their programming knowledge.

A structured prompt format was chosen instead of an open-ended prompt because it produces more predictable and organized results. This makes it easier for the frontend to display

the information and ensures that users receive comprehensive feedback every time they submit code. By combining prompt engineering with the reasoning capabilities of Gemini, DevMentor AI delivers meaningful code reviews and personalized learning assistance in a consistent and user-friendly manner.

5. Trade-Offs Considered

During the development of DevMentor AI, several design decisions and trade-offs were considered to balance development time, system performance, and functionality. One of the most important decisions was choosing between developing a custom machine learning model and integrating an existing Large Language Model through an API. Training a custom model would have provided greater control over the analysis process and allowed for customization specific to code review tasks. However, it would have required a large dataset, significant computational resources, and extensive training time. To ensure rapid development and high-quality results, the Gemini API was selected as the AI engine for the project.

Using the Gemini API provided several advantages. It enabled the application to leverage state-of-the-art language understanding and code analysis capabilities without the need for complex model training. The API supports multiple programming languages and can generate detailed reviews, complexity analysis, and learning recommendations. This allowed the project to focus on solving the user problem rather than spending time building and training an AI model from scratch.

However, this decision also introduced certain limitations. Since the application depends on an external AI service, it requires an active internet connection to function. The system is also subject to API rate limits, quota restrictions, and service availability. During testing, quota limitations occasionally restricted the number of requests that could be processed. Additionally, API-based solutions may introduce latency compared to locally hosted models.

Another trade-off involved keeping the backend architecture simple by using Flask instead of more complex frameworks. Flask was chosen because it is lightweight, easy to develop with, and suitable for a prototype application. While larger frameworks may provide additional features and scalability, Flask allowed faster development and easier integration with the Gemini API.

Overall, the selected design choices prioritize simplicity, rapid development, and practical usability. Although the system depends on external AI services and internet connectivity, the benefits of high-quality AI-powered code analysis and faster implementation outweigh these limitations for the scope of this project.

6. Conclusion

DevMentor AI demonstrates how Large Language Models can be effectively integrated into software development and learning tools. By combining a modern web-based interface, a Flask backend, and the Gemini API, the system provides intelligent code reviews, complexity analysis, and personalized learning recommendations in real time. The application helps students

and early-career developers identify coding mistakes, improve code quality, and strengthen their programming knowledge through actionable feedback.

The project follows a clear Input → Process → Output workflow and utilizes prompt engineering to generate structured and meaningful responses from the AI model. Through features such as code analysis, bug detection, learning suggestions, file upload support, and code statistics, DevMentor AI offers a practical solution to a real-world problem faced by many learners.

Although the system depends on external AI services and is subject to API limitations, it successfully demonstrates the potential of AI-powered developer tools. Future enhancements may include support for additional programming languages, interview question generation, user authentication, review history, and offline analysis capabilities. Overall, DevMentor AI showcases how artificial intelligence can be leveraged to improve coding productivity, accelerate learning, and provide personalized guidance to aspiring developers.